```
...
@login_required
def room_editor(request, venue_id):
    venue = get_object_or_404(Venue, id=venue_id,
        userprofile=request.user.userprofile)

    data = {
        "venue": venue,
    }

    return render(request, "room_editor.html", data)
```

Only logged-in users should be able to edit a venue.

Find the given venue and ensure the logged-in user is associated with it.

The HTML for the room editor is broken down into two parts: the main page and the partial that renders a room. You saw a similar pattern with the search musicians feature: the partial that needs to be rendered is often also included into the main page as well. The following listing contains the main page.

Listing 12.13 The main room editor page

```
<!-- RiffMates/templates/room_editor.html -->
{% extends "base.html" %}

{% block content %}
  <h1>Venue: {{venue.name}}</h1>

  <ul>
    {% for room in venue.room_set.all %}
      {% include "partials/show_room.html" %}
    {% empty %}
      <li><i>No rooms for this venue</i> </li>
    {% endfor %}
  </ul>

{% endblock content %}
```

The same subtemplate is used as the partial for another view.

The corresponding subtemplate for each room is shown in the following listing.

Listing 12.14 The main room editor page

```
<!-- RiffMates/templates/partials/show_room.html -->
<li id="room-row-{{room.id}}">
  {{room.name }}   
  <a
      hx-get="{% url 'edit_room_form' room.id %}"
      hx-target="#room-row-{{room.id}}"
      style="cursor: pointer; text-decoration: underline"
    > (Edit) </a>
</li>
```

Display the room's name.

The Edit link uses an HTMX GET to retrieve the room-editing form.

When GET gets invoked, replace the entire <li> tag.

As the <a> tag has no href, the browser doesn't style it; add the underline and pointer cursor back to it.

With the two templates in place, you can test the room editor view. Don't forget to register the `room_editor()` as a route in bands/urls.py, and make sure you include the